

Chapter 4th - Recurrent Neural Network for NLP

Dr. Shiv Shakti Shrivastava

Professor

Department of CSE, PIT

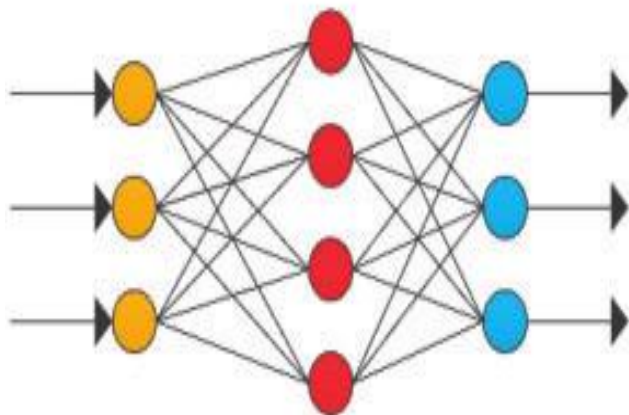
Content

1. Recurrent Neural Network for NLP: What is RNN?
2. Difference between Feedforward and Recurrent Neural Networks
3. Basics of Recurrent, Neural Networks
4. Natural Language Processing & Recurrent Neural Networks
5. RNN's Mechanism, Training RNN
6. Meta meaning of Hidden State of RNN
7. Tuning RNN, Long Short-Term
8. Memory Networks, Sequence to Sequence Models
9. Advance Sequence to Sequence Models

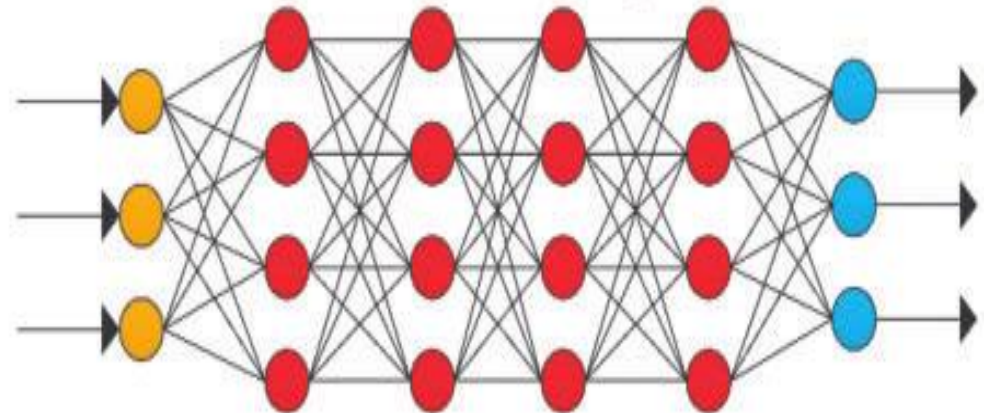
Recurrent Neural Network(RNN)

- Recurrent Neural Network(RNN) is a type of [Neural Network](#) where the output from the previous step is fed as input to the current step.
- In traditional neural networks, all the inputs and outputs are independent of each other, but in cases when it is required to predict the next word of a sentence, the previous words are required and hence there is a need to remember the previous words.
- Thus RNN came into existence, which solved this issue with the help of a Hidden Layer.

Neural Network



Deep Learning



● Input Layer ● Hidden Layer ● Output Layer

Recurrent Neural Network(RNN)...

- The main and most important feature of RNN is its **Hidden state**, which remembers some information about a sequence.
- The state is also referred to as *Memory State* since it remembers the previous input to the network.
- It uses the same parameters for each input as it performs the same task on all the inputs or hidden layers to produce the output.
- This reduces the complexity of parameters, unlike other neural networks.

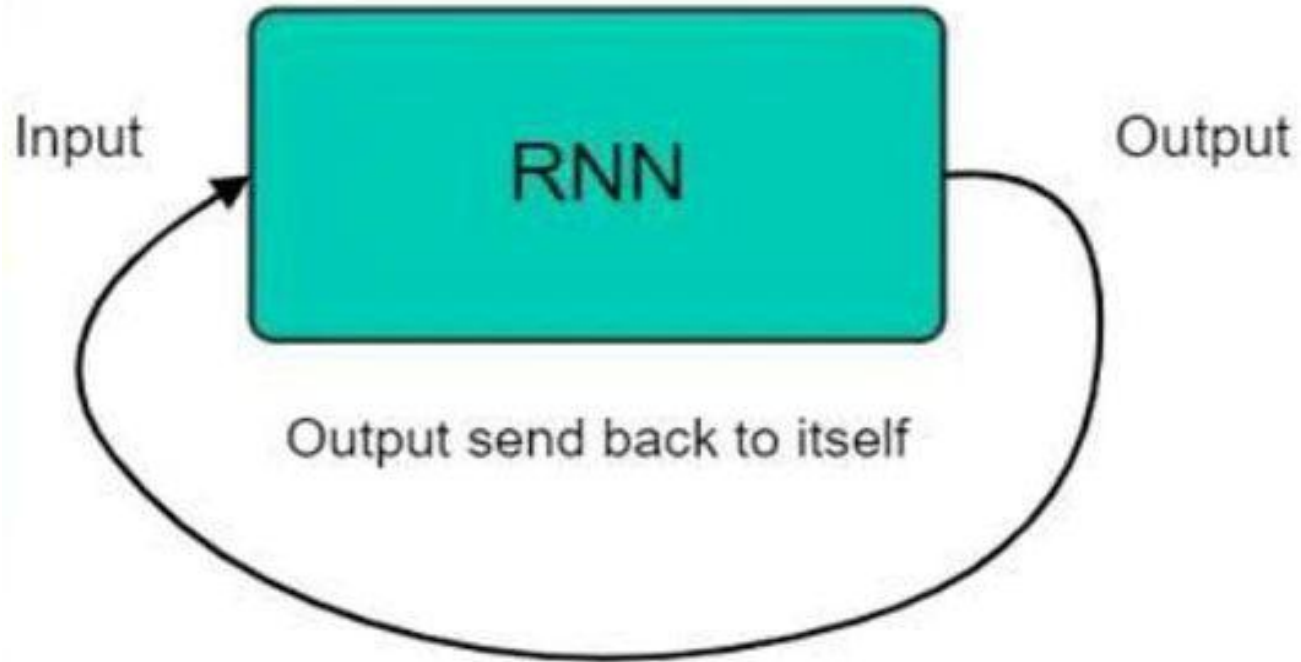


Fig.: RNN

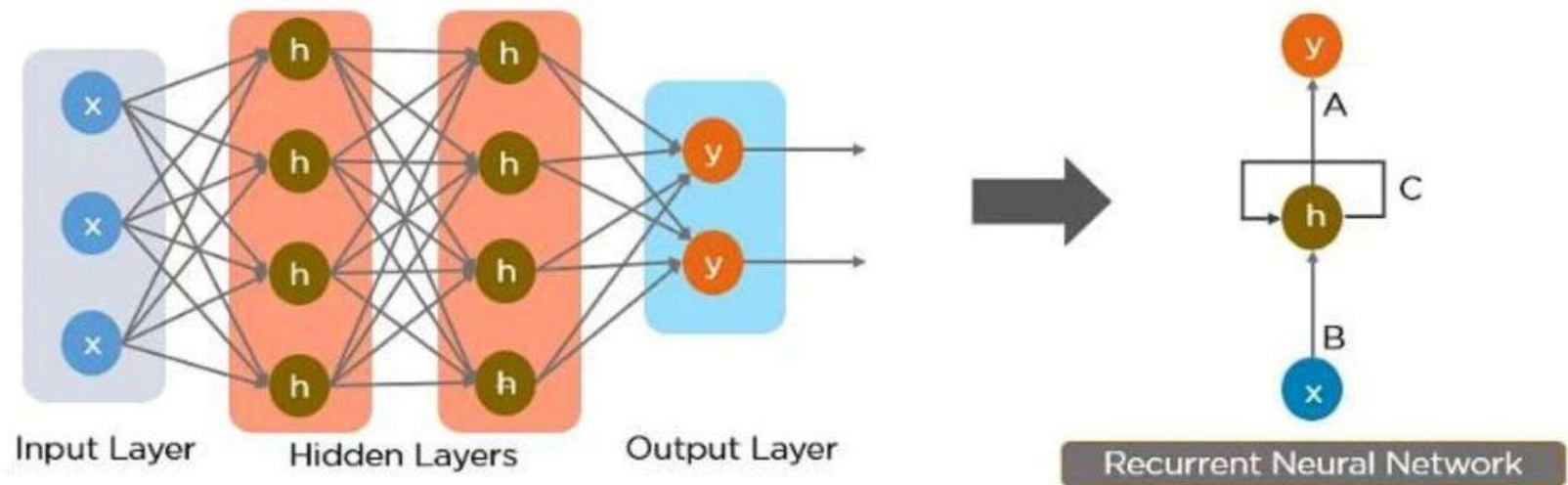
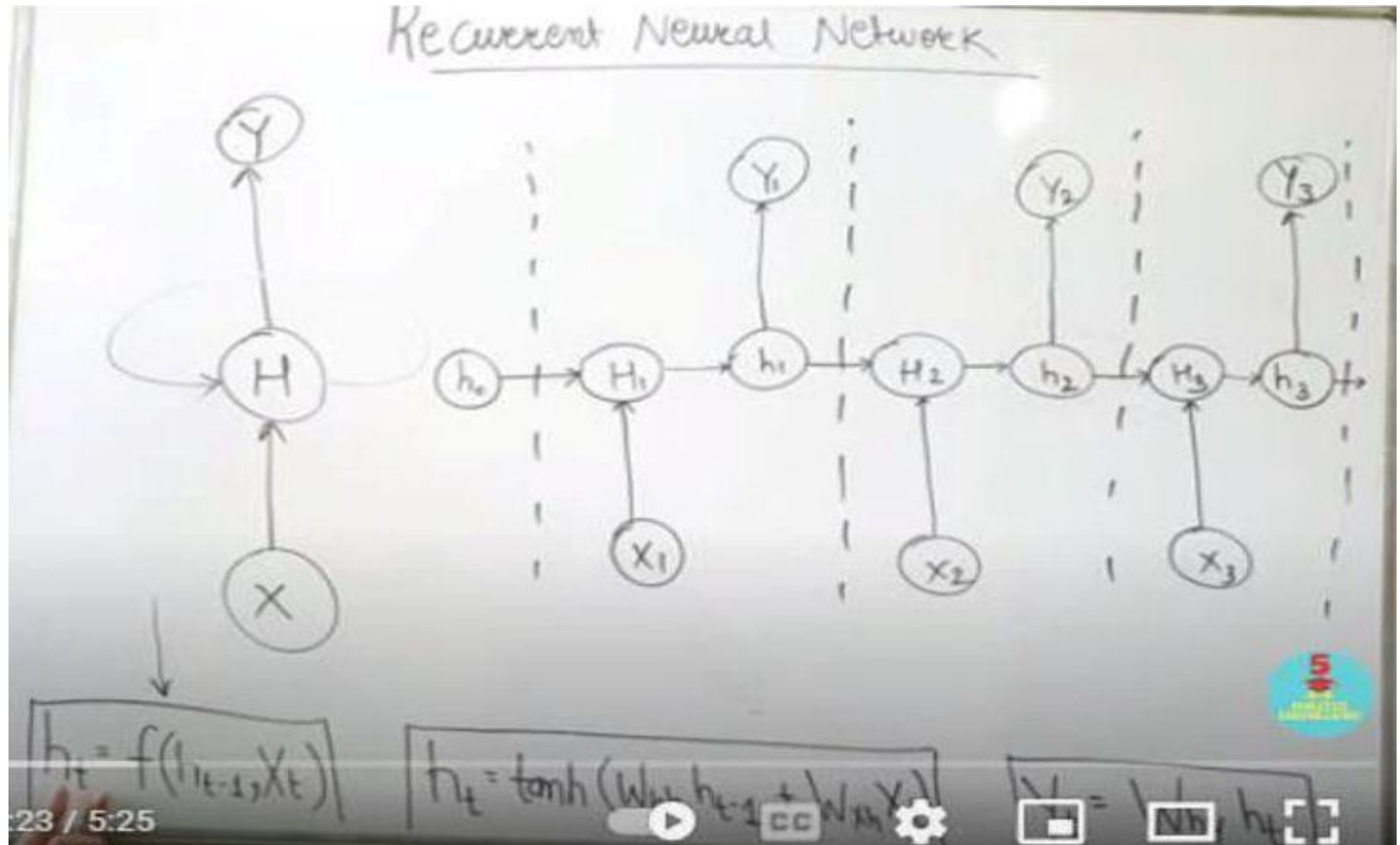
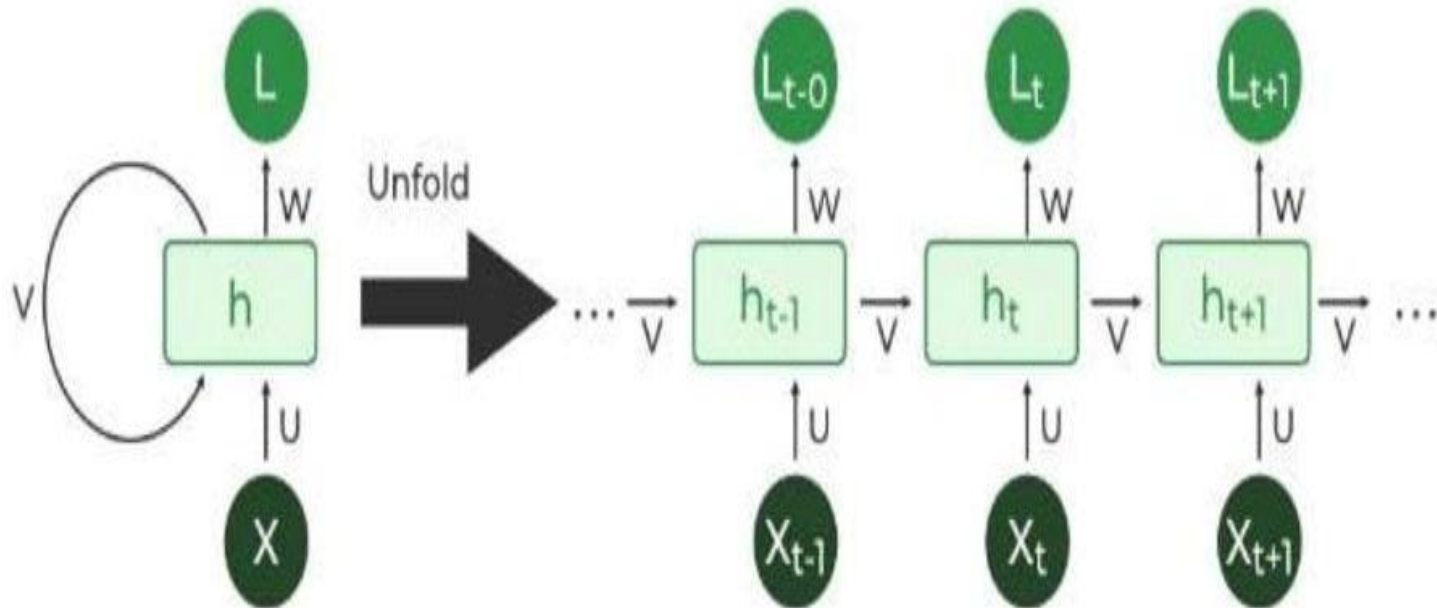


Fig: Simple Recurrent Neural Network

The nodes in different layers of the neural network are compressed to form a single layer of recurrent neural networks. A, B, and C are the parameters of the network.





How RNN works

- The Recurrent Neural Network consists of multiple fixed activation function units, one for each time step.
- Each unit has an internal state which is called the hidden state of the unit.
- This hidden state signifies the past knowledge that the network currently holds at a given time step.
- This hidden state is updated at every time step to signify the change in the knowledge of the network about the past.
- The hidden state is updated using the following recurrence relation:-

The formula for calculating the current state:

$$H_t = f(H_{t-1}, x_t)$$

where:

h_t -> current state

h_{t-1} -> previous state

x_t -> input state

- Formula for applying Activation function(tanh):

$$H_t = \tanh(w_{hh}H_{t-1} + W_{xh} x_t)$$

where:

w_{hh} -> weight at recurrent neuron

w_{xh} -> weight at input neuron

- The formula for calculating output:

$$Y_t = w_{hy}t$$

Where:

Y_t -> output

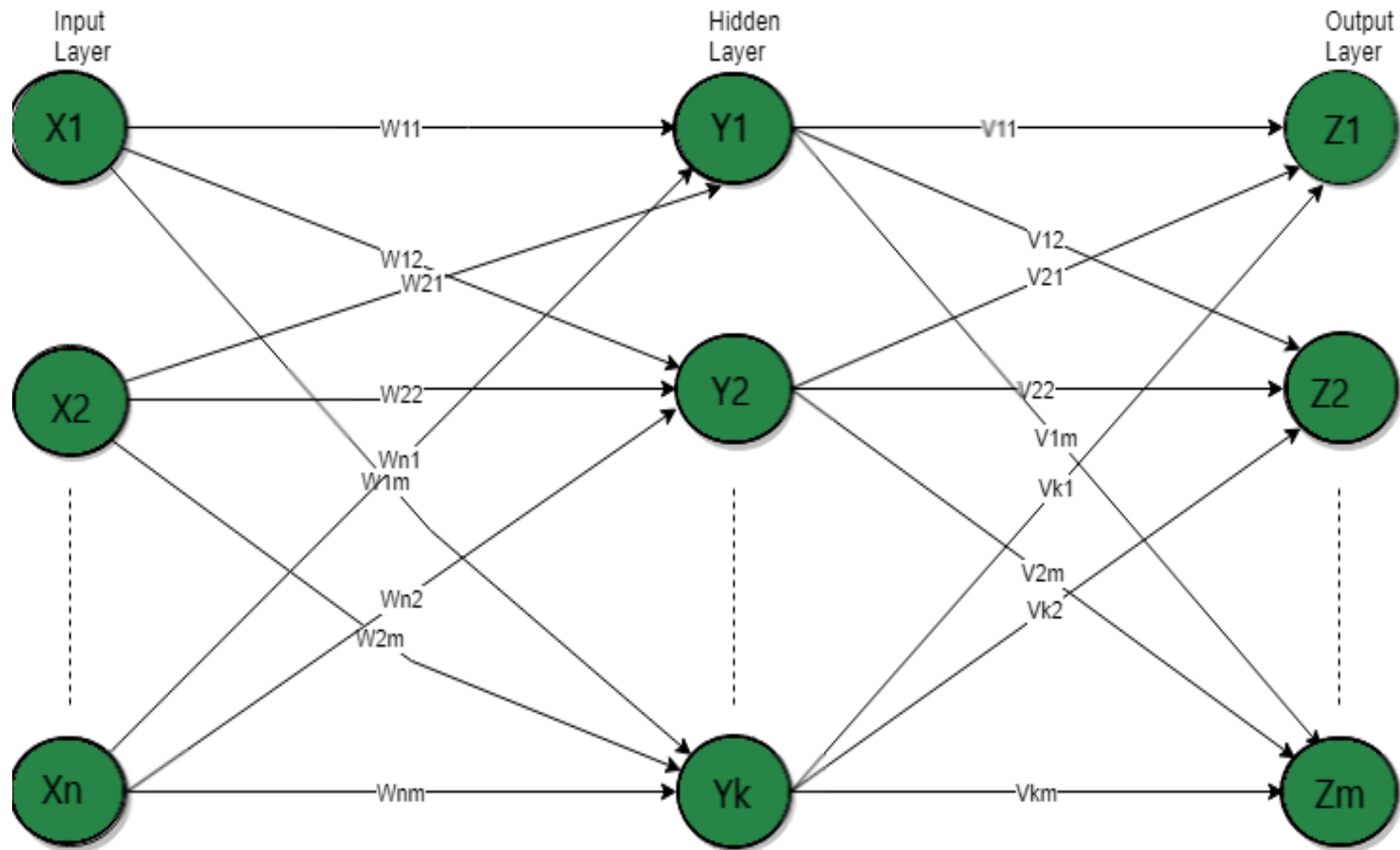
W_{hy} -> weight at output layer

- These parameters are updated using Backpropagation.
- However, since RNN works on sequential data here we use an updated backpropagation which is known as Backpropagation through time.

Difference between Feedforward and Recurrent Neural Networks

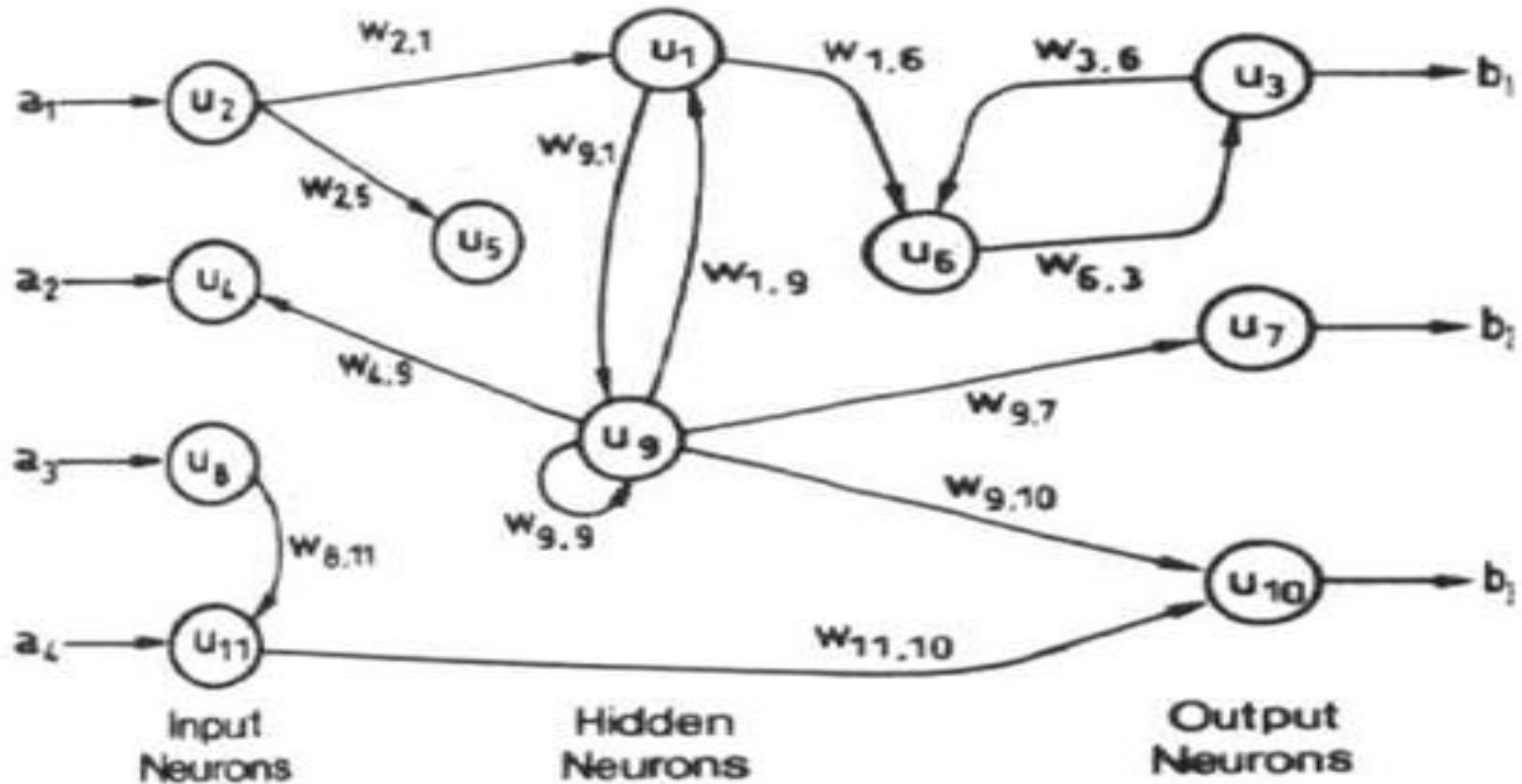
Feedforward: ANNs allow signals to travel one way only: from input to output.

- There are no feedback (loops); *i.e.*, the output of any layer does not affect that same layer. Feed-forward ANNs tend to be straightforward networks that associate inputs with outputs.
- They are extensively used in pattern recognition. This type of organization is also referred to as bottom-up or top-down.



Feedback: (or recurrent or interactive) networks can have signals traveling in both directions by introducing loops in the network.

- Feedback networks are powerful and can get extremely complicated.
- Computations derived from earlier input are fed back into the network, which gives them a kind of memory.
- Feedback networks are **dynamic**; their **'state'** is changing continuously until they reach an equilibrium point.
- They remain at the equilibrium point until the input changes and a new equilibrium needs to be found.



Difference Between FFNN & RNN

Comparison Attribute	Feed-forward Neural Networks	Recurrent Neural Networks
Signal flow direction	Forward only	Bidirectional
Delay introduced	No	Yes
Complexity	Low	High
Neuron independence in the same layer	Yes	No
Speed	High	slow
Commonly used for	Pattern recognition, speech recognition, and character recognition	Language translation, speech-to-text conversion, and robotic control

Recurrent Neural Network(RNN):

- Recurrent neural networks lead to an increase in complexity (RNN).
- They keep track of the output from processing nodes and include it in the model (they only did not pass the information in one direction).
- This is how the model is claimed to learn to anticipate the outcome of a layer.
- Each node in the RNN model serves as a memory cell, continuing calculation and operation execution when a network predicts something incorrectly.
- Through backpropagation, the system self-learns and keeps trying to generate the correct forecast.

Pros:

- An RNN retains every piece of knowledge throughout time. Only the ability to remember past inputs makes it helpful for time series prediction. Long Short-Term Memory is the term for this.
- Convolutional layers and recurrent neural networks are combined with increasing the effective pixel neighborhood.

Cons:

- Problems with gradient disappearing and explosions.
- It is exceedingly tough to train an RNN.
- If Tanh or ReLu are used as the activation function, it cannot process long sequences.

Back propagation method:

- This term is very important because we will discuss about vanishing gradient in the next section which depends on back-propagation.

error = actual output – output calculated by the model

Eq. 2

*change ∈ weight = learning rate * (rate of change ∈ $\frac{\text{error}}{\text{weight}}$)*

Eq. 3

new weight = old weight + change ∈ weight

Eq. 4

- Back-propagation means there is feedback from output to input which is to minimize the error between the actual output and output predicted by the model.
- With the help of back propagation method, we can update the weights again so that output will match the actual output.
- For this change in weight is calculated using the equation (3).
- Now the new weights are updated using the old weights and change in weight which is calculated in the equation 4.

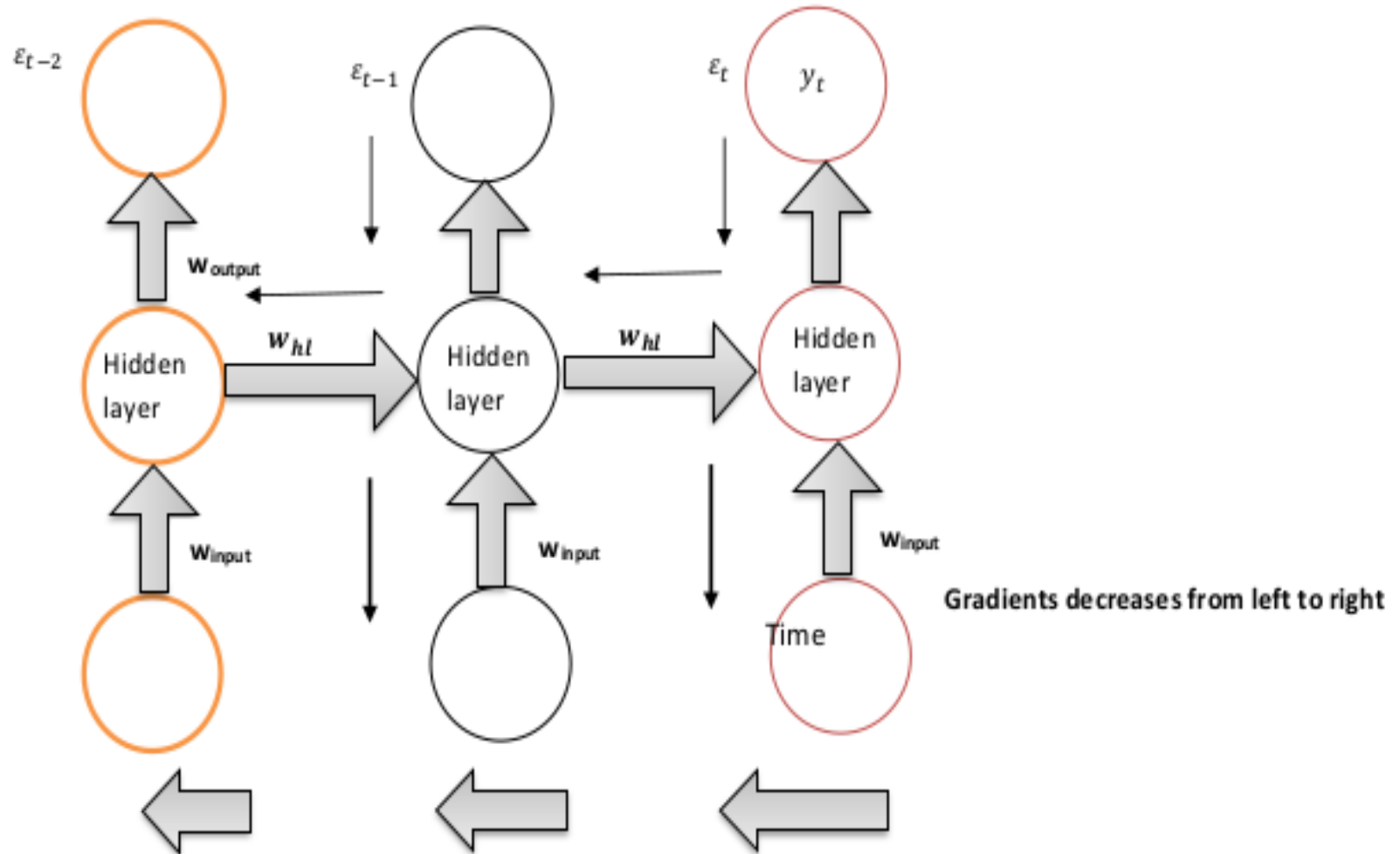
Problem in RNN- Vanishing Gradient?

- Let us understand what is **vanishing gradient** as well as what is **exploding gradient**.
- Understanding of these two terms will help to understand the problem in RNN. For these we need to use equation 3 and 4. In vanishing gradient $d(e)/d(w) \ll 1$.
- This will make change on weight negligible. If you calculate the new weight using the equation (4), there will negligibly change in the weight which leads to vanishing gradient problem.

Problem in RNN- Vanishing Gradient?

- Now let us understand what is **exploding gradient** problem. If $d(e)/d(w) \gg 1$, in such case change in weight becomes quite large which makes the new weight quite large as compared to the old weight.
- This will leads to exploding gradient problem. **There are two factors that effect the magnitude of gradient. One is weight and other is activation function.**
- Gradients for deeper layers are calculated as the product of many gradients (activation function).

- If you want to calculate the error ϵ It will depend to the previous states i.e. weights associated with $e(t-2)$ and $e(t-1)$ So when gradient $(d(\epsilon)/d(w))$ is backpropagated back, all the previous weights (previous layers) needs to be updated.
- Problem is that when gradient is calculated it will increase towards the cost function ϵ_t but decreases while moving away from it.
- During training gradient associated with $e(t-2)$ has no effect on the output $(y(t))$.



Understanding vanishing gradient in RNN in these key points

- Rate of change of error with respect to weight is propagated back to the time step network to minimize the cost function.
- Gradient in backward direction are calculated as product of derivatives of activation function whose magnitude is high in the hidden layers to the right and less in the hidden layers in the left.
- So when moving in the left direction gradient decreases and hence gradient is vanished.

Basics of Recurrent Neural Networks

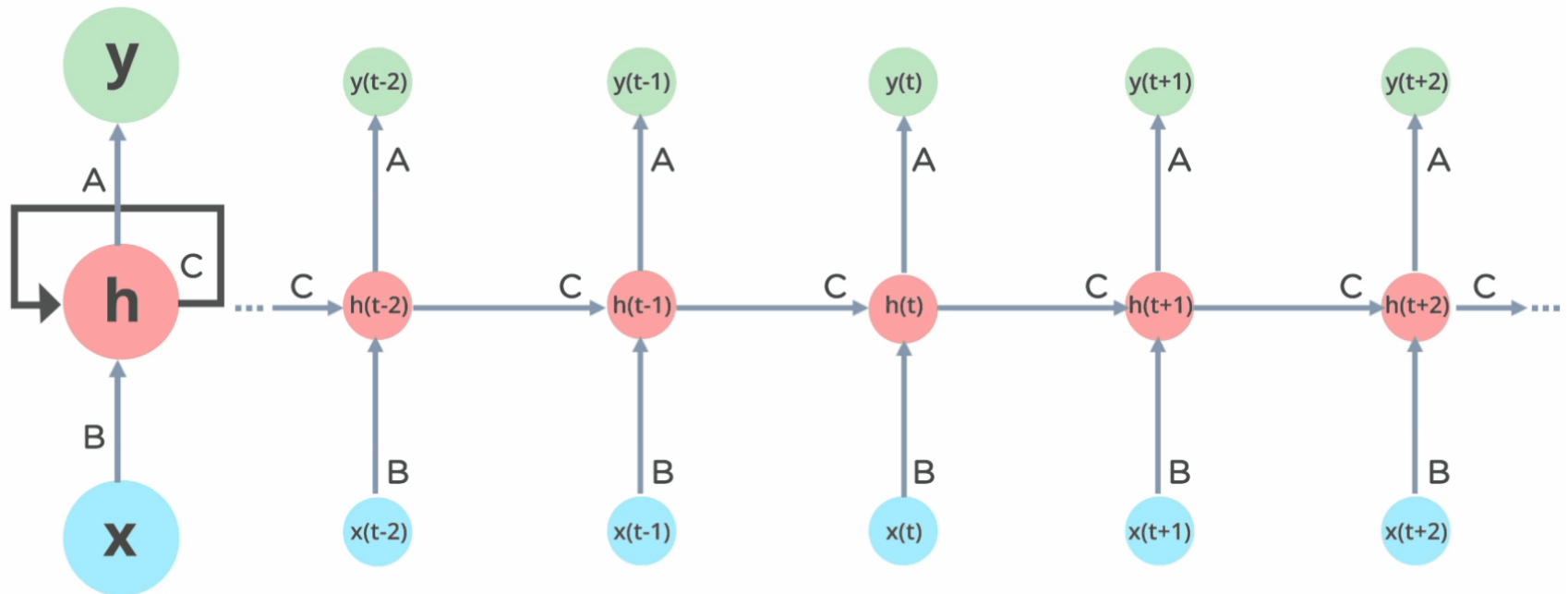
- Recurrent Neural Networks (RNNs) are a type of artificial neural network designed for handling sequential data, such as time series, natural language, and audio signals.
- They are particularly effective in tasks where the order and context of the data matter.
- RNNs have an internal memory that allows them to maintain information about previous inputs in order to make decisions or predictions based on the entire sequence.

Steps-Algorithm to Train an RNN

1. We have to input a specific dataset or object with data.
2. The network will accept a scenario and perform some random initialized variables.
3. It then computes a predicted result.
4. There will be a difference produced (as an error) while calculating the actual result generated and the expected value.
5. We propagate the model through the same path where the variables get adjusted to identify the error.
6. Repeat steps (1) to (5) until we are confident that the variables for getting the desired output relate to what we are expecting.
7. Such an algorithm creates a systematic prediction by using these variables to obtain further unseen input.

How Does Recurrent Neural Networks Work?

- In Recurrent Neural networks, the information cycles through a loop to the middle-hidden layer.



- The input layer 'x' takes in the input to the neural network and processes it and passes it onto the middle layer.
- The middle layer 'h' can consist of multiple hidden layers, each with its own activation functions and weights and biases.
- If you have a neural network where the various parameters of different hidden layers are not affected by the previous layer, ie: the neural network does not have memory, then you can use a recurrent neural network.
- The Recurrent Neural Network will standardize the different activation functions and weights and biases so that each hidden layer has the same parameters. Then, instead of creating multiple hidden layers, it will create one and loop over it as many times as required.

Applications of Recurrent Neural Networks-

1. Image Captioning- RNNs are used to caption an image by analyzing the activities present.

2. Time Series Prediction -

- Any time series problem, like predicting the prices of stocks in a particular month, can be solved using an RNN.

3. Natural Language Processing-

- Text mining and Sentiment analysis can be carried out using an RNN for Natural Language Processing (NLP).

4. Machine Translation-

- Given an input in one language, RNNs can be used to translate the input into different languages as output.

Advantages of RNN

1. Ability To Handle Variable-Length Sequences-

- RNNs are designed to handle input sequences of variable length, which makes them well-suited for tasks such as speech recognition, natural language processing, and time series analysis.

2. Memory Of Past Inputs-

- RNNs have a memory of past inputs, which allows them to capture information about the context of the input sequence. This makes them useful for tasks such as language modeling, where the meaning of a word depends on the context in which it appears.

3. Parameter Sharing-

- RNNs share the same set of parameters across all time steps, which reduces the number of parameters that need to be learned and can lead to better generalization.

4. Non-Linear Mapping-

- RNNs use non-linear activation functions, which allows them to learn complex, non-linear mappings between inputs and outputs.

5. Sequential Processing-

- RNNs process input sequences sequentially, which makes them computationally efficient and easy to parallelize.

6. Flexibility-

- RNNs can be adapted to a wide range of tasks and input types, including text, speech, and image sequences.

7. Improved Accuracy-

- RNNs have been shown to achieve state-of-the-art performance on a variety of sequence modeling tasks, including language modeling, speech recognition, and machine translation.

Limitations of RNN-

- Due to multiple recurrent execution flows within the algorithm, the entire computation becomes slow.
- It is often difficult to train an RNN model for beginners and even experts.
- If we use activation functions "relu" or "tanh", it becomes problematic and tough to process long-sized sequences within the model.
- RNN is prone to various real-time problems like gradient vanishing and exploding.

Basics of Recurrent Neural Networks

- Recurrent Neural Networks (RNNs) are a type of artificial neural network designed for handling sequential data, such as time series, natural language, and audio signals.
- They are particularly effective in tasks where the order and context of the data matter.
- RNNs have an internal memory that allows them to maintain information about previous inputs in order to make decisions or predictions based on the entire sequence.
- The basic idea behind RNNs is to introduce loops within the neural network architecture, which allows information to be passed from one step of the sequence to the next.
- This loop allows RNNs to maintain a hidden state that captures relevant information from previous steps and combines it with the current input to produce an output and update the hidden state for the next step.

Basic components and functioning of Recurrent Neural Networks:

1. **Input and Output Layers:** Similar to other neural networks, an RNN consists of input and output layers.

The input can be a sequence of data points, where each point is represented as a vector, and the output can be a single value or a sequence of values.

2. **Recurrent Connections:** The key feature of RNNs is the recurrent connections, which allow the output of a particular step to be fed back into the network as input for the next step.

This connection enables the network to maintain memory of previous steps.

- 3. Hidden State:** The hidden state is a vector that gets updated at each time step and captures information from previous steps. It serves as the "memory" of the RNN and influences the current step's output. The hidden state at time step t is used as an input for time step $t+1$.
- 4. Activation Function:** RNNs use an activation function (often tanh or ReLU) to process the combined input at each time step. This function introduces non-linearity and allows the network to capture complex relationships in the data.
- 5. Training:** RNNs are trained using techniques like backpropagation through time (BPTT). The gradients are computed through the entire sequence, essentially unrolling the RNN for a fixed number of steps, and then updating the weights using gradient descent optimization algorithms.

However, traditional RNNs suffer from some limitations, such as difficulty in capturing long-range dependencies due to the vanishing gradient problem.

- To address this, variations of RNNs have been developed, including Long Short-Term Memory (LSTM) networks and Gated Recurrent Units (GRUs).
- These architectures include mechanisms to better preserve and update information over longer sequences.

- LSTM and GRU networks maintain a similar structure to RNNs but introduce specialized gates that control the flow of information into and out of the hidden state, allowing them to better capture longer dependencies in the data.
- In summary, Recurrent Neural Networks are designed to handle sequential data by maintaining a hidden state that captures information from previous steps.
- While traditional RNNs have limitations in capturing long-range dependencies, LSTM and GRU networks address these issues with specialized gating mechanisms.

- Imagine a basic RNN as a series of interconnected nodes, each representing a time step in a sequence. The RNN processes input data at each time step and maintains a hidden state that carries information from previous time steps. Here's a textual representation of the basic structure:

```
1. Input(t-1) -> Hidden(t-1) -+
      |
2. Input(t)  -> Hidden(t) ----+--> Output(t)
      |
3. Input(t+1) -> Hidden(t+1) -+
```


From the previous page table:

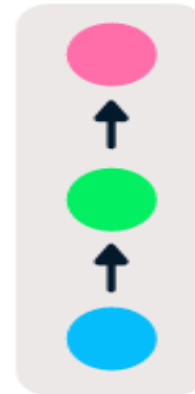
1. **Input(t)**-represents the input data at time step **t**.
2. **Hidden(t)**- represents the hidden state of the RNN at time step **t**, which carries information from both the previous hidden state (Hidden(t-1)) and the current input (Input(t)).
3. **Output(t)**- is the output generated by the RNN at time step **t**.
 - At each time step, the RNN calculates the hidden state using the current input and the previous hidden state.
 - This hidden state is then used to produce the output at that time step.
 - The hidden state is updated in a way that allows the network to capture sequential dependencies in the data.

Types of Recurrent Neural Networks-

- There are four types of Recurrent Neural Networks:
 1. One to One
 2. One to Many
 3. Many to One
 4. Many to Many

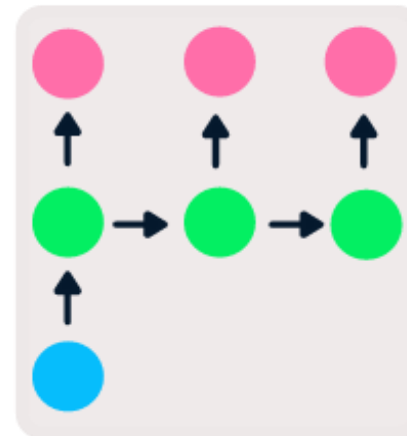
1. One-to-one is a simple neural network. It is commonly used for machine learning problems that have a single input and output.

One to One



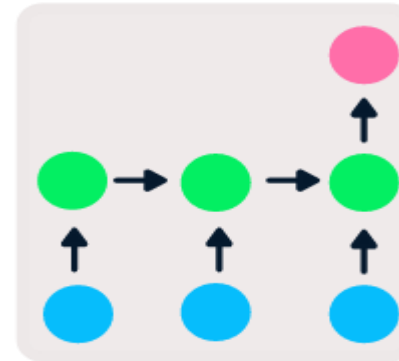
2. One-to-many has a single input and multiple outputs. This is used for generating image captions.

One to Many



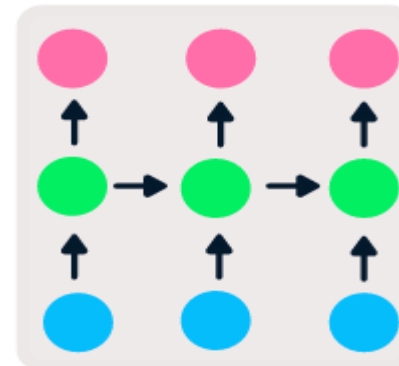
3. Many-to-one takes a sequence of multiple inputs and predicts a single output. It is popular in sentiment classification, where the input is text and the output is a category.

Many to One



4. Many-to-many takes multiple inputs and outputs. The most common application is machine translation.

Many to Many



RNN Advanced Architectures-

- The simple RNN repeating modules have a basic structure with a single tanh layer.
- RNN simple structure suffers from short memory, where it struggles to retain previous time step information in larger sequential data.
- These problems can easily be solved by long short term memory (LSTM) and gated recurrent unit (GRU), as they are capable of remembering long periods of information.

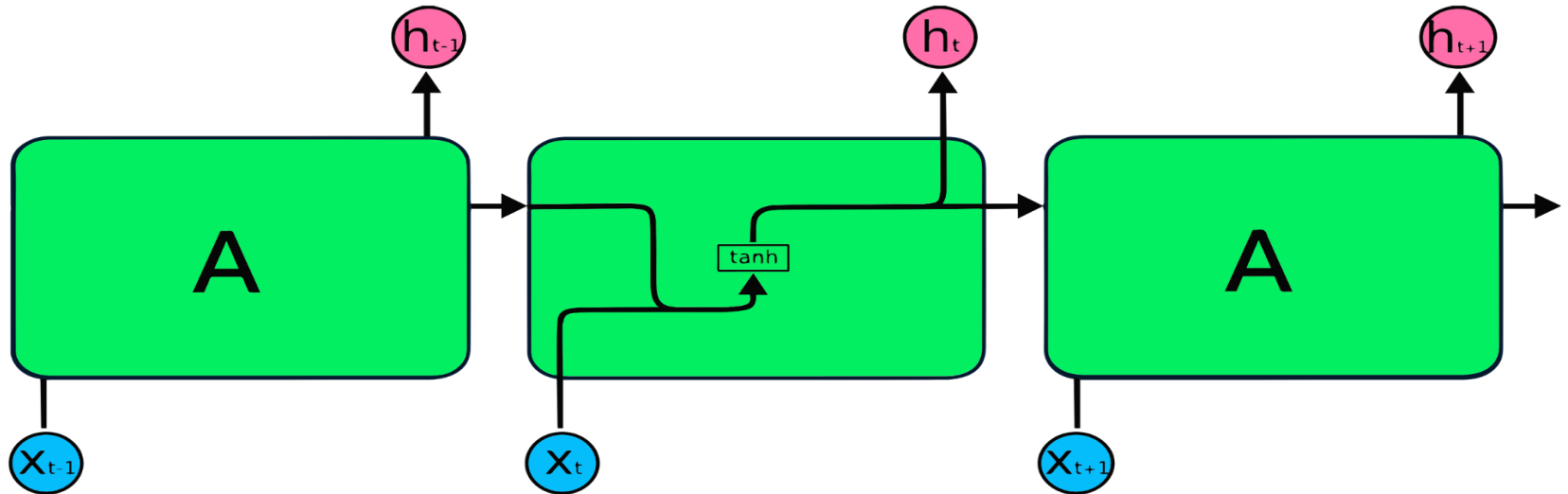
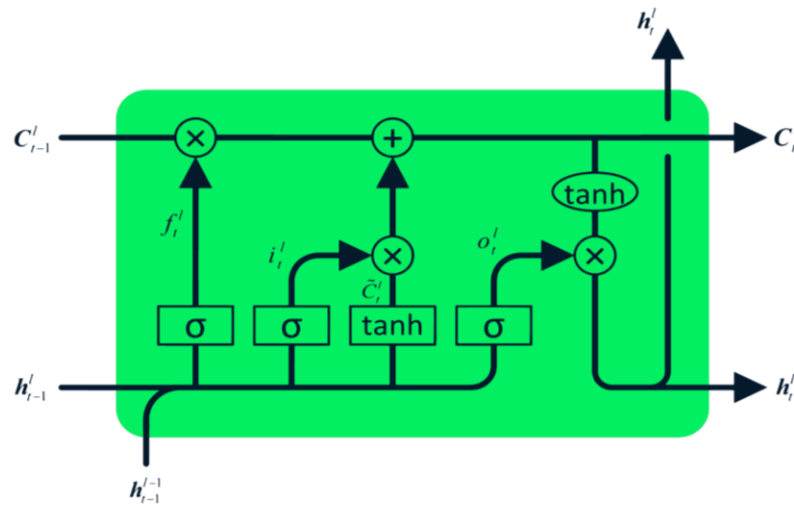


Fig.: Simple RNN Cell

Long Short-Term Memory (LSTM)-

- The Long Short Term Memory (LSTM) is the advanced type of RNN, which was designed to prevent both decaying and exploding gradient problems.
- Just like RNN, LSTM has repeating modules, but the structure is different. Instead of having a single layer of tanh, LSTM has four interacting layers that communicate with each other.
- This four-layered structure helps LSTM retain long-term memory and can be used in several sequential problems including machine translation, speech synthesis, speech recognition, and handwriting recognition. You can gain hands-on experience in LSTM by following the guide: [Python LSTM for Stock Predictions](#).

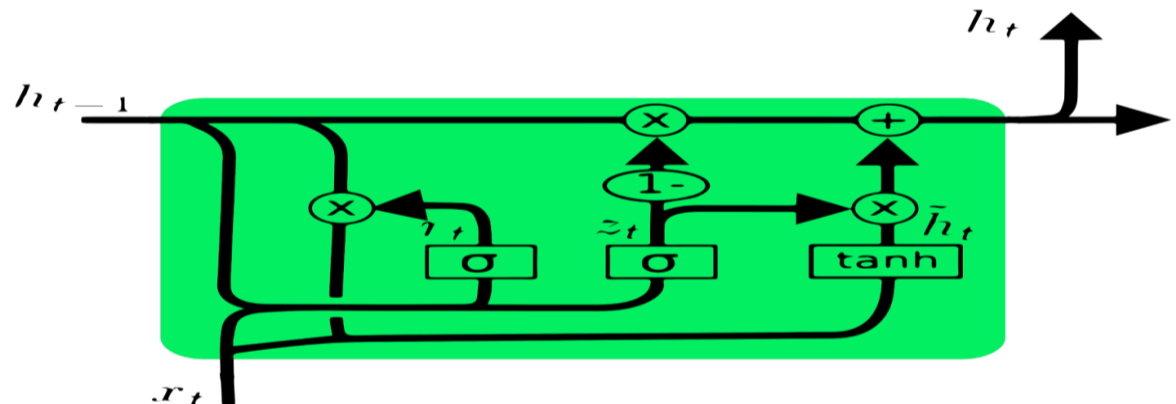
Fig.: LSTM Cell



Gated Recurrent Unit (GRU)-

- The gated recurrent unit (GRU) is a variation of LSTM as both have design similarities, and in some cases, they produce similar results.
- GRU uses an **update gate** and **reset gate** to solve the vanishing gradient problem.
- These gates decide what information is important and pass it to the output.
- The gates can be trained to store information from long ago, without vanishing over time or removing irrelevant information

Fig.: GRU Cell



Sequence to Sequence Models

- Sequence-to-Sequence (Seq2Seq) models are a type of neural network architecture used in natural language processing and other sequence-based tasks.
- They are particularly popular for tasks that involve taking a sequence of data as input and producing a sequence of data as output.
- Seq2Seq models are based on recurrent neural networks (RNNs) and have been instrumental in various applications, including machine translation, text summarization, chatbots, and speech recognition

Here's an overview of how Seq2Seq models work:

1. Encoder-Decoder Architecture:

- Seq2Seq models consist of two main components: an encoder and a decoder.
- The encoder takes the input sequence and encodes it into a fixed-length context vector, also known as the "thought vector."
- The decoder then takes this context vector and generates the output sequence, one step at a time.

2. Variable-Length Sequences:

- Seq2Seq models can handle input and output sequences of different lengths, making them suitable for tasks like machine translation where the length of the source and target sentences can vary.

3. Recurrent Neural Networks (RNNs):

- Traditionally, RNNs were used as the building blocks for both the encoder and decoder.
- The encoder processes the input sequence step by step, updating its hidden state at each time step.
- The final hidden state of the encoder becomes the context vector passed to the decoder.
- The decoder also uses an RNN to generate the output sequence, one token at a time, while updating its hidden state.

4. Attention Mechanism:

- To address issues with long-range dependencies, attention mechanisms were introduced in Seq2Seq models.
- Attention allows the decoder to focus on specific parts of the input sequence when generating each output token.
- This mechanism has significantly improved the model's ability to capture context.

5. Training with Teacher Forcing:

- During training, Seq2Seq models often use a technique called "teacher forcing."
- Instead of using the previously generated token as input at each decoding step, the true target token from the training data is used.
- This helps with training stability but can lead to exposure bias, where the model performs well during training but poorly during inference.

Applications of Seq2Seq Models:

1. **Machine Translation:** Seq2Seq models have been used in machine translation systems like Google's Neural Machine Translation (GNMT).
2. **Text Summarization:** They can generate concise summaries of longer texts.
3. **Speech Recognition:** Seq2Seq models have been applied to convert spoken language into written text.
4. **Chatbots:** Seq2Seq models are used in conversational AI to generate responses to user queries.
5. **Image Captioning:** Combining Convolutional Neural Networks (CNNs) with Seq2Seq models allows generating textual descriptions for images.
6. **Mathematical Problem Solving:** Seq2Seq models can solve mathematical problems when given equations as input.

Overall, Seq2Seq models have been a significant advancement in the field of sequence processing and have enabled the development of various applications that require handling sequences of data, especially in the domain of natural language understanding and generation.

Advance Sequence to Sequence Models

- Advanced Sequence-to-Sequence (Seq2Seq) models are an extension of the basic Seq2Seq architecture, incorporating various enhancements and improvements to tackle more complex natural language processing and sequence generation tasks.
- These models are designed to address some of the limitations of traditional Seq2Seq models.
- Here are some advanced techniques and models in the Seq2Seq paradigm:

1. Attention Mechanisms:

- Attention mechanisms improve the basic Seq2Seq model's ability to focus on relevant parts of the input sequence when generating the output sequence.
- Popular attention mechanisms include Bahdanau Attention and Luong Attention.
- Transformers, a breakthrough architecture, heavily rely on attention mechanisms for sequence modeling.

2. Transformer-Based Models:

- Transformers have become the state-of-the-art architecture for various NLP tasks, including machine translation, text summarization, and more.
- Transformer models, such as BERT (Bidirectional Encoder Representations from Transformers) and GPT (Generative Pre-trained Transformer), have achieved remarkable results in understanding and generating natural language.

3. BERT (Bidirectional Encoder Representations from Transformers):

- BERT is a pre-trained transformer-based model designed for various NLP tasks, including question answering, sentiment analysis, and named entity recognition.
- It uses a masked language modeling objective to capture bidirectional context during pre-training.

4. GPT (Generative Pre-trained Transformer):

- GPT is a series of transformer-based models developed for text generation tasks.
- GPT-3, the largest model in the series, has demonstrated impressive natural language generation capabilities across a wide range of applications.

5. Multimodal Seq2Seq Models:

- Some tasks involve both text and other modalities, such as images or audio.
- Multimodal Seq2Seq models combine information from multiple modalities to generate sequences or captions for complex tasks like image captioning.

6. Copy Mechanisms:

- Copy mechanisms enable Seq2Seq models to copy words or phrases directly from the input sequence to the output sequence.
- They are useful in tasks like text summarization and handling out-of-vocabulary words.

7. Reinforcement Learning for Seq2Seq:

- Reinforcement learning techniques are used to fine-tune Seq2Seq models for tasks like dialogue generation and machine translation.
- These techniques help improve the quality of generated sequences.

8. Transfer Learning and Pre-training:

- Transfer learning involves pre-training Seq2Seq models on large datasets and fine-tuning them for specific downstream tasks.
- This approach has significantly improved the performance of Seq2Seq models in various applications.

9. Beam Search and Decoding Strategies:

- Advanced decoding strategies like beam search and nucleus sampling improve the quality of generated sequences.
- They are crucial for generating coherent and contextually relevant output.

10. Custom Architectures:

- Researchers often customize Seq2Seq architectures to suit specific tasks. Architectures like T5 (Text-to-Text Transfer Transformer) treat all NLP tasks as text-to-text problems, simplifying model design.
- These advanced Seq2Seq techniques and models have pushed the boundaries of natural language understanding and generation. Researchers continue to explore innovative approaches to make Seq2Seq models more effective in handling complex language tasks, paving the way for breakthroughs in various NLP applications.

Tuning RNN

- Tuning Recurrent Neural Networks (RNNs) is a critical step in developing effective models for sequence-based tasks, including natural language processing, speech recognition, time series forecasting, and more.
- Proper tuning involves optimizing various hyperparameters and architectural choices to improve the model's performance.
- Here are the key aspects to consider when tuning RNNs:

1. **Architecture Selection:**

Choose the appropriate RNN architecture for your task. Common choices include vanilla RNN, Long Short-Term Memory (LSTM), and Gated Recurrent Unit (GRU).

Experiment with advanced variants like bidirectional RNNs, stacked RNN layers, or even hybrid models like LSTM-CNN.

2. **Hidden Layer Size:**

Adjust the number of hidden units or neurons in the RNN layers. Larger hidden layers can capture more complex patterns but may require more data and computational resources.

3. **Sequence Length:**

Determine the sequence length or time steps used as input to the RNN. Longer sequences can capture more context but may lead to increased training time and memory requirements.

4. Batch Size:

Batch size affects how many sequences are processed in parallel during training. Smaller batches may introduce noise, while larger batches can speed up training but consume more memory.

5. Learning Rate:

The learning rate determines the step size during gradient descent optimization. It needs to be carefully chosen to ensure convergence without overshooting the optimal solution.

Consider using learning rate schedules or adaptive optimizers like Adam, RMSprop, or Adagrad.

6. Regularization Techniques:

- Apply regularization techniques to prevent overfitting, such as dropout and L2 regularization on weights.
- Experiment with variational dropout for recurrent layers, which can be more effective for RNNs.

7. Initialization:

- Properly initialize weights, as it can affect convergence and training speed. Common techniques include Xavier/Glorot initialization and He initialization.

8. Gradient Clipping:

- Gradient explosion is a common issue in RNN training. Implement gradient clipping to limit the gradient values during training to prevent this problem.

9. Recurrent Dropout:

- Apply dropout specifically to the recurrent connections in LSTM or GRU layers to prevent overfitting.

10. Early Stopping:

- Monitor validation loss during training and stop training when it starts to increase (indicating overfitting). Early stopping helps prevent the model from learning noise in the data.

11. Batch Normalization:

- Batch normalization can stabilize training by normalizing activations within each mini-batch.

12. Hyperparameter Search:

- Conduct a systematic search for the best hyperparameters using techniques like grid search or random search.
- Use cross-validation to evaluate different combinations of hyperparameters.

13. Attention Mechanisms:

- Consider incorporating attention mechanisms into your RNN architecture to improve the model's ability to focus on relevant parts of the input sequence.

14. Ensemble Learning:

- Combine predictions from multiple RNN models to improve overall performance. Bagging or boosting can be effective strategies.

15. Transfer Learning:

- For some tasks, transfer learning from pre-trained RNN models or embeddings (e.g., word embeddings like Word2Vec or GloVe) can provide a significant boost in performance.

16. Hardware Acceleration:

- Utilize GPUs or TPUs for training RNN models, as they significantly speed up training.

17. Monitoring and Visualization:

- Monitor training progress with validation metrics and visualize training curves to identify issues such as underfitting or overfitting.

- Overall, the Tuning RNNs requires experimentation and a good understanding of the specific task and dataset.
- It often involves a combination of these techniques to achieve the best model performance. Keep in mind that hyperparameter tuning can be a time-consuming process, but it is essential for achieving state-of-the-art results.

Parul[®]
University

NAAC
GRADE **A++**



<https://paruluniversity.ac.in/>

